

Initializers of objects with automatic and dynamic storage duration have funny inconsistencies

Abstract

This paper talks about the differences of initializers of objects with automatic vs. dynamic storage duration; in particular, the difference of initializing an object of a type vs. initializing a pointer to that type with a new-expression. See the thread starting at <http://lists.isocpp.org/ext/2017/12/3419.php>.

The gist of the problem is that these are different:

```
double* p = new double[]{1,2,3}; // not ok
double a[] {1,2,3}; // ok
```

A different inconsistency:

```
double** pp = new double*{nullptr}; // ok
double* p = double*{nullptr}; // not ok
```

The issue explained

An innocent user might expect that if you can write `T x{something};`, you could also write `T* x = new T{something};`, and vice versa. In both of the examples in the abstract, that is not the case:

- For the first example, the straightforward array variable declaration is ok, but the pointer initialized by new is not, because new requires the expression in the brackets
- For the second example, as Timur Doumler explains it in the mailing list thread, "in a new-declarator the * is always parsed as part of the type - in a regular initialiser it's instead parsed as part of an expression."

The second example may be beyond fixing. Making the * be part of the type rather than part of the expression is certainly a breaking change.

For the first example, Jens Maurer provided an explanation why it doesn't work:

Level 1 answer:

Because the expression inside the square brackets is mandatory for a new-expression; see 8.5.2.4 [expr.new] p1. That situation is unchanged since at least C++03.

Level 2 answer:

When braced-init-lists were added to /new-initializer/ in the course of uniform initialization, the rule about deducing the size of the array from the number of initializers was never extended to the new-expression case.

Level 3 answer:

I don't know of a fundamental reason why we can't make this work. Papers welcome.

It seems to me that a combination of adding deduction to a new-expression and then removing the requirement for the expression inside square brackets should fix the problem in the first example.

A more detailed solution

Hey, I just wrote this paper so that we can talk about the issue, the actual technical solution is left as an exercise for readers. :)